UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ingeniería en Tecnologías de la Información e Innovación Digital

S3

Búsqueda local: Hill Climbing y Simulated Annealing

Actividad Parcial 2 · Fundamentos de Inteligencia Artificial

Tema 2.3 · Optimización de la ubicación de centros de acopio

Alumno	Rogerio Aguilar Bello
Matrícula	24305118
Grupo / cuatrimestre	171 · 7.º
Docente	Mtro. Javier García Lara
Tipo de entrega	Individual · 50 pts
Fecha de entrega	22 de octubre de 2026
Referencias del tema	Wirsansky (2020), cap. 2
Repositorio GitHub	https://github.com/RogerNika/Rogerio-Aguilar-Bello-FIA

Plus de esta actividad: laboratorio de sensibilidad (paisaje de costo, mapa de calor de $T0 \times \alpha$ y efecto del paso η) para explicar por qué funciona cada algoritmo.

Capítulo 1. Planteamiento del problema

1.1 Contexto

Muchos problemas de decisión tienen demasiadas soluciones posibles para revisarlas todas. Los algoritmos de búsqueda local parten de una solución y la mejoran paso a paso moviéndose a soluciones vecinas; son rápidos y usan poca memoria, pero pueden quedar atrapados en óptimos locales. Esta actividad compara dos de ellos, Hill Climbing y Simulated Annealing, en un problema logístico real.

1.2 Problema

Una cooperativa debe ubicar **dos centros de acopio** para recibir producto de 12 comunidades del oriente del Estado de México y Morelos. El objetivo es minimizar la distancia total ponderada por la demanda (ton·km por semana) entre cada comunidad y su centro más cercano. La función objetivo no es convexa: tiene mínimos locales, por lo que el resultado depende de cómo se busque.

1.3 Datos del problema

Las coordenadas son aproximadas (cabeceras municipales, proyectadas a km respecto de Amecameca) y las demandas semanales son **ilustrativas**, definidas para el ejercicio.

Comunidad	Demanda (ton/sem)	Comunidad	Demanda (ton/sem)
Amecameca	14	Ixtapaluca	18
Ozumba	8	Juchitepec	5
Atlautla	6	Tenango del Aire	4
Ecatzingo	4	Cocotitlán	3
Tlalmanalco	9	Texcoco	16
Chalco	20	Cuautla	12

Capítulo 2. Objetivos

2.1 Objetivo general

Comparar Hill Climbing y Simulated Annealing mediante su implementación en Python y un experimento con 30 semillas aleatorias para determinar cuál encuentra mejores ubicaciones de dos centros de acopio con menor variabilidad.

2.2 Objetivos específicos

- **Formular** la función objetivo, el vecindario y los parámetros del problema **definiendo** variables y restricciones **para** poder aplicar búsqueda local.
- **Implementar** Hill Climbing y Simulated Annealing en Python **usando** un mismo vecindario gaussiano **para** compararlos en igualdad de condiciones.
- **Medir** costo final, tasa de éxito, tiempo y curva de convergencia **mediante** 30 ejecuciones pareadas **para** evaluar calidad y estabilidad.
- **Analizar** la sensibilidad a la temperatura inicial, el enfriamiento y el paso **variando** cada parámetro **para** justificar la configuración elegida.

Capítulo 3. Fundamentos

Búsqueda local. Mantiene una única solución actual y evalúa soluciones vecinas con una función objetivo; no guarda el camino recorrido (Russell y Norvig, 2020). **Hill Climbing** se mueve al vecino solo si mejora la función objetivo; se detiene cuando ningún vecino mejora, por lo que puede quedar en un óptimo local o en una meseta. **Simulated Annealing** añade el criterio de Metropolis: acepta un vecino peor con probabilidad $\exp(-\Delta/T)$, donde Δ es el empeoramiento y T la temperatura; T desciende con el tiempo (enfriamiento geométrico $T \leftarrow \alpha \cdot T$), de modo que al inicio explora y al final se comporta como Hill Climbing (Kirkpatrick et al., 1983; Metropolis et al., 1953).

3.1 Parámetros

- **N:** número de iteraciones (presupuesto de evaluaciones).
- **η:** paso del vecindario (desviación estándar, en km, de la perturbación gaussiana).
- **T_0 y α:** temperatura inicial y factor de enfriamiento de Simulated Annealing. La temperatura final es $T_0 \cdot \alpha^N$.

Referencia de comparación. Como no se conoce el óptimo exacto, se usa el mejor valor hallado por evolución diferencial (Storn y Price, 1997), un optimizador global de SciPy (Virtanen et al., 2020).

Capítulo 4. Metodología y procedimiento

- Modelar el problema: variables $x = (x_A, y_A, x_B, y_B)$ en km y objetivo $f(x) = \Sigma\, w_i \cdot \min(d(p_i, A), d(p_i, B))$.
- Definir el vecino: $x' = x + N(0, \eta^2)$ en cada coordenada, recortado a la caja de búsqueda.
- Implementar Hill Climbing, Hill Climbing con 10 reinicios y Simulated Annealing con el mismo presupuesto (N = 3,000 evaluaciones).
- Ejecutar 30 semillas; en cada una, Hill Climbing y Simulated Annealing parten del mismo punto inicial (diseño pareado).
- Registrar la curva del mejor costo, el costo final, la tasa de éxito (costo ≤ 1 % sobre la referencia) y el tiempo.
- Barrer T_0, α y η para estudiar la sensibilidad (plus) y redactar el análisis.

Capítulo 5. Desarrollo: formulación y configuración

Variables	Posición de dos centros: $x = (x_A, y_A, x_B, y_B)$, en km.
Función objetivo	$f(x) = \Sigma\, w_i \cdot \min(d(p_i, A), d(p_i, B))$; se minimiza (ton·km/semana).
Restricciones	Los centros deben quedar dentro de la caja que rodea a las comunidades (± 5 km).
Vecino	Perturbación gaussiana de desviación η en las 4 coordenadas.
Parámetros	N = 3000; η = 3 km; $T_0 = 60$; α = 0.998 (T final ≈ 0.15); 30 semillas.
Referencia	1318.72 ton·km (evolución diferencial).

Capítulo 6. Resultados

Método	Media	Desv. est.	Mejor	Peor	Éxito (≤ 1 % ref.)	Tiempo (ms)
HC	1377.7	116.8	1322.3	1642.0	83%	43.4
HC reinicios	1326.2	1.6	1322.8	1330.7	100%	43.4
SA	1325.7	2.0	1321.9	1328.9	100%	44.4

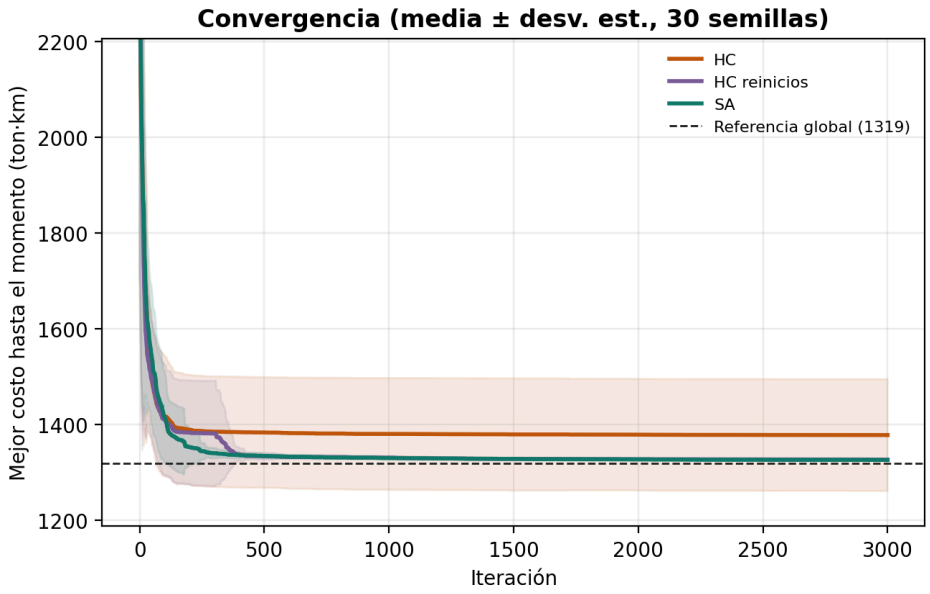


Figura 1. Mejor costo hasta cada iteración (media ± desviación estándar de 30 semillas).

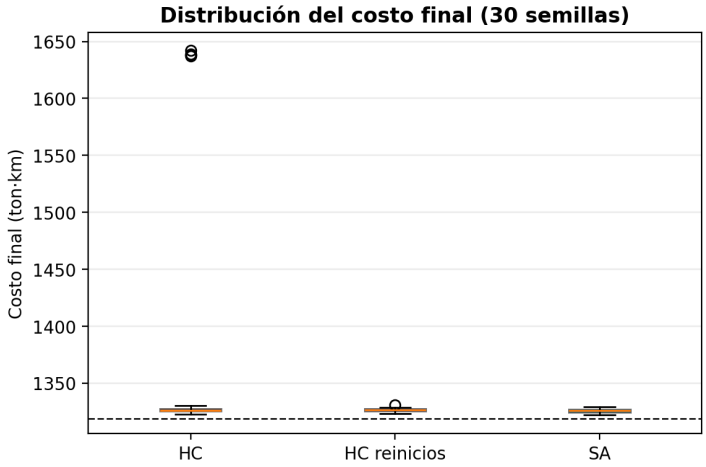


Figura 2. Distribución del costo final por método.

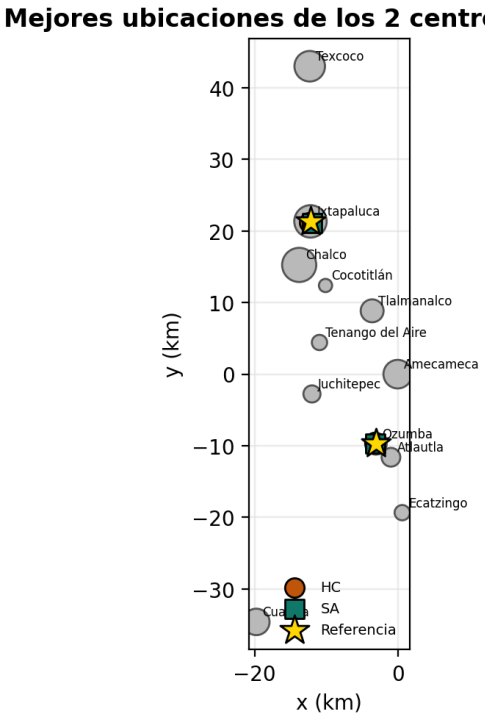


Figura 3. Mejores ubicaciones halladas frente a la referencia global (estrellas).

Capítulo 7. Código Python

Archivo `fia_s3_busqueda_local.py` (requiere `datos_edomex.py`). Pip: numpy, scipy, matplotlib.

```
"""
FIA - Actividad Parcial 2 - Sección 3 (2.3 Búsqueda local)
Hill Climbing y Simulated Annealing para ubicar 2 centros de acopio que minimicen la distancia
ponderada total a 12 comunidades (problema tipo k-medianas continuo, no convexo => mínimos locales).

  Variables   : x = (xA, yA, xB, yB) en km (4 dimensiones)
  Objetivo    : f(x) = sum_i w_i * min(dist(p_i, A), dist(p_i, B))   (minimizar, unidades: ton-km/semana)
  Vecino      : x' = x + N(0, eta^2) por coordenada, recortado a la caja de búsqueda
  Parámetros  : iteraciones N, paso eta (km), temperatura inicial T0 y factor de enfriamiento alpha (SA)

Requisitos: pip install numpy scipy matplotlib
Uso:        python fia_s3_busqueda_local.py        (genera figuras y resultados_s3.json)
"""
import json, time
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.optimize import differential_evolution
from datos_edomex import a_km

# ------------------------------------------------------------------------------ datos
COMUNIDADES = {  # demanda semanal ILUSTRATIVA (toneladas)
    "Amecameca": 14, "Ozumba": 8, "Atlautla": 6, "Ecatzingo": 4, "Tlalmanalco": 9, "Chalco": 20,
    "Ixtapaluca": 18, "Juchitepec": 5, "Tenango del Aire": 4, "Cocotitlán": 3, "Texcoco": 16, "Cuautla": 12,
}
NOMBRES = list(COMUNIDADES)
PTS = np.array([a_km(n) for n in NOMBRES])           # (12, 2) en km
W = np.array([COMUNIDADES[n] for n in NOMBRES], float)
LO, HI = PTS.min(axis=0) - 5, PTS.max(axis=0) + 5     # caja de búsqueda
LO4, HI4 = np.tile(LO, 2), np.tile(HI, 2)


def f(x):
    """Función objetivo: distancia ponderada de cada comunidad al centro más cercano."""
    A, B = x[:2], x[2:]
```

```
    d = np.minimum(np.linalg.norm(PTS - A, axis=1), np.linalg.norm(PTS - B, axis=1))
    return float(np.sum(W * d))


def vecino(x, eta, rng):
    return np.clip(x + rng.normal(0.0, eta, 4), LO4, HI4)


# ------------------------------------------------------------------------------ algoritmos
def hill_climbing(x0, n_iter=3000, eta=3.0, rng=None):
    """Ascenso de colina estocástico (versión de descenso): acepta solo si mejora."""
    x, fx = x0.copy(), f(x0); curva = [fx]
    for _ in range(n_iter):
        y = vecino(x, eta, rng); fy = f(y)
        if fy < fx:
            x, fx = y, fy
        curva.append(fx)
    return x, fx, curva


def hill_climbing_reinicios(n_iter=3000, eta=3.0, reinicios=10, rng=None):
    """HC con reinicios aleatorios: el presupuesto de iteraciones se reparte entre los reinicios."""
    mejor, fm, curva = None, np.inf, []
    por = n_iter // reinicios
    for _ in range(reinicios):
        x, fx, c = hill_climbing(rng.uniform(LO4, HI4), por, eta, rng)
        for v in c[1:]:
            fm = min(fm, v); curva.append(fm)
        if fx <= fm: mejor, fm = x, fx
    return mejor, fm, [curva[0]] + curva


def simulated_annealing(x0, n_iter=3000, eta=3.0, T0=60.0, alpha=0.998, rng=None):
    """Recocido simulado: acepta peores con prob. exp(-delta/T); enfriamiento geométrico T <- alpha*T."""
    x, fx = x0.copy(), f(x0); mejor, fb = x.copy(), fx; T = T0; curva = [fb]
    for _ in range(n_iter):
        y = vecino(x, eta, rng); fy = f(y); delta = fy - fx
        if delta < 0 or rng.random() < np.exp(-delta / T):
            x, fx = y, fy
            if fx < fb: mejor, fb = x.copy(), fx
        T *= alpha; curva.append(fb)
    return mejor, fb, curva


# ------------------------------------------------------------------------------ experimentos
def referencia():
    """Mejor valor conocido con optimización global (evolución diferencial) como patrón de comparación."""
    r = differential_evolution(f, list(zip(LO4, HI4)), seed=0, tol=1e-10, maxiter=500, polish=True)
    return r.x, r.fun


def experimento(n_semillas=30, n_iter=3000, eta=3.0, T0=60.0, alpha=0.998):
    res = {"HC": [], "HC reinicios": [], "SA": []}; curvas = {k: [] for k in res}; tiempos = {k: [] for k in res}
    soluciones = {}
    for s in range(n_semillas):
        rng0 = np.random.default_rng(1000 + s); x0 = rng0.uniform(LO4, HI4)
        for nombre, fn in (("HC", lambda r: hill_climbing(x0, n_iter, eta, r)),
                           ("HC reinicios", lambda r: hill_climbing_reinicios(n_iter, eta, 10, r)),
                           ("SA", lambda r: simulated_annealing(x0, n_iter, eta, T0, alpha, r))):
            t = time.perf_counter(); x, fx, c = fn(np.random.default_rng(5000 + s));
tiempos[nombre].append(time.perf_counter() - t)
            res[nombre].append(fx); curvas[nombre].append(c)
            if fx == min(res[nombre]): soluciones[nombre] = x
    return res, curvas, tiempos, soluciones


def sensibilidad(Ts=(5, 20, 60, 150, 400), alphas=(0.990, 0.995, 0.998, 0.9995), n_semillas=12, n_iter=3000):
    M = np.zeros((len(Ts), len(alphas)))
    for i, T0 in enumerate(Ts):
        for j, a in enumerate(alphas):
            vals = []
            for s in range(n_semillas):
                x0 = np.random.default_rng(1000 + s).uniform(LO4, HI4)
                vals.append(simulated_annealing(x0, n_iter, 3.0, T0, a, np.random.default_rng(7000 + s))[1])
            M[i, j] = np.mean(vals)
    return Ts, alphas, M
```

```
def sensibilidad_eta(etas=(0.5, 1, 2, 3, 6, 12), n_semillas=15, n_iter=3000):
    out = {"HC": [], "SA": []}
    for e in etas:
        h, a = [], []
        for s in range(n_semillas):
            x0 = np.random.default_rng(1000 + s).uniform(LO4, HI4)
            h.append(hill_climbing(x0, n_iter, e, np.random.default_rng(5000 + s))[1])
            a.append(simulated_annealing(x0, n_iter, e, 60.0, 0.998, np.random.default_rng(5000 + s))[1])
        out["HC"].append(float(np.mean(h))); out["SA"].append(float(np.mean(a)))
    return etas, out


# ---------------------------------------------------------------------------- gráficas
COL = {"HC": "#C2570C", "HC reinicios": "#7A5C99", "SA": "#0F7B6C"}


def graficas(res, curvas, ref_x, ref_f, sol, Hm, ruta=""):
    fig, ax = plt.subplots(figsize=(7.4, 4.4), dpi=200)
    for k, c in curvas.items():
        C = np.array(c); m, s = C.mean(0), C.std(0)
        ax.plot(m, color=COL[k], lw=2, label=k); ax.fill_between(range(len(m)), m - s, m + s, color=COL[k],
alpha=.12)
    ax.axhline(ref_f, color="#222", ls="--", lw=1, label=f"Referencia global ({ref_f:.0f})")
    ax.set_xlabel("Iteración"); ax.set_ylabel("Mejor costo hasta el momento (ton·km)"); ax.set_ylim(ref_f * 0.9,
np.array(curvas["HC"]).mean(0)[5] * 1.05)
    ax.set_title("Convergencia (media ± desv. est., 30 semillas)", fontweight="bold"); ax.legend(frameon=False,
fontsize=8); ax.grid(alpha=.25)
    fig.savefig(ruta + "s3_convergencia.png", bbox_inches="tight"); plt.close(fig)

    fig, ax = plt.subplots(figsize=(6.4, 4.2), dpi=200)
    bp = ax.boxplot([res[k] for k in res], tick_labels=list(res), patch_artist=True)
    for p, k in zip(bp["boxes"], res): p.set_facecolor(COL[k]); p.set_alpha(.55)
    ax.axhline(ref_f, color="#222", ls="--", lw=1); ax.set_ylabel("Costo final (ton·km)")
    ax.set_title("Distribución del costo final (30 semillas)", fontweight="bold"); ax.grid(alpha=.25, axis="y")
    fig.savefig(ruta + "s3_boxplot.png", bbox_inches="tight"); plt.close(fig)

    fig, ax = plt.subplots(figsize=(6.4, 5.4), dpi=200)
    ax.scatter(PTS[:, 0], PTS[:, 1], s=W * 14, c="#BBB", edgecolors="#555", zorder=2)
    for n, p in zip(NOMBRES, PTS): ax.annotate(n, p, fontsize=6, xytext=(3, 3), textcoords="offset points")
    for k, m in (("HC", "o"), ("SA", "s")):
        for j in (0, 2): ax.scatter(*sol[k][j:j + 2], marker=m, s=90, color=COL[k], edgecolors="k", zorder=4,
label=k if j == 0 else None)
    for j in (0, 2): ax.scatter(*ref_x[j:j + 2], marker="*", s=220, color="gold", edgecolors="k", zorder=5,
label="Referencia" if j == 0 else None)
    ax.set_xlabel("x (km)"); ax.set_ylabel("y (km)"); ax.set_aspect("equal"); ax.legend(fontsize=7, frameon=False)
    ax.set_title("Mejores ubicaciones de los 2 centros de acopio", fontweight="bold"); ax.grid(alpha=.2)
    fig.savefig(ruta + "s3_mapa.png", bbox_inches="tight"); plt.close(fig)

    Ts, alphas, M = Hm
    fig, ax = plt.subplots(figsize=(6.2, 4.2), dpi=200)
    im = ax.imshow(M, cmap="YlGn_r", aspect="auto")
    ax.set_xticks(range(len(alphas)), [str(a) for a in alphas]); ax.set_yticks(range(len(Ts)), [str(t) for t in
Ts])
    ax.set_xlabel("alpha (enfriamiento)"); ax.set_ylabel("T0 (temperatura inicial)")
    for i in range(len(Ts)):
        for j in range(len(alphas)): ax.text(j, i, f"{M[i, j]:.0f}", ha="center", va="center", fontsize=8)
    fig.colorbar(im, label="Costo medio"); ax.set_title("Plus: sensibilidad de SA a T0 y alpha", fontweight="bold")
    fig.savefig(ruta + "s3_heatmap.png", bbox_inches="tight"); plt.close(fig)

    # paisaje de costo: centro A variable con B fijo en la referencia
    xs = np.linspace(LO[0], HI[0], 160); ys = np.linspace(LO[1], HI[1], 160); Z = np.zeros((160, 160))
    for i, yv in enumerate(ys):
        for j, xv in enumerate(xs): Z[i, j] = f(np.array([xv, yv, *ref_x[2:]]))
    fig, ax = plt.subplots(figsize=(6.4, 5.2), dpi=200)
    cs = ax.contourf(xs, ys, Z, 28, cmap="viridis_r"); fig.colorbar(cs, label="f(x) con B fijo")
    ax.scatter(PTS[:, 0], PTS[:, 1], c="white", s=14, edgecolors="k"); ax.scatter(*ref_x[2:], marker="*", s=200,
c="gold", edgecolors="k")
    ax.set_xlabel("xA (km)"); ax.set_ylabel("yA (km)"); ax.set_title("Plus: paisaje de costo (mínimos locales)",
fontweight="bold")
    fig.savefig(ruta + "s3_paisaje.png", bbox_inches="tight"); plt.close(fig)

    return None


def graf_eta(etas, out, ruta=""):
    fig, ax = plt.subplots(figsize=(6.4, 3.8), dpi=200)
```

```
    for k in out: ax.plot(etas, out[k], "o-", color=COL[k], label=k)
    ax.set_xscale("log"); ax.set_xlabel("Paso eta (km, escala log)"); ax.set_ylabel("Costo medio final");
ax.legend(frameon=False); ax.grid(alpha=.25)
    ax.set_title("Plus: efecto del paso eta", fontweight="bold"); fig.savefig(ruta + "s3_eta.png",
bbox_inches="tight"); plt.close(fig)


if __name__ == "__main__":
    ref_x, ref_f = referencia()
    print(f"Referencia global (evolución diferencial): f = {ref_f:.2f}")
    res, curvas, tiempos, sol = experimento()
    resumen = {}
    for k, v in res.items():
        v = np.array(v)
        resumen[k] = dict(media=float(v.mean()), desv=float(v.std()), mejor=float(v.min()), peor=float(v.max()),
                          exito_1pct=float(np.mean(v <= ref_f * 1.01)), tiempo_s=float(np.mean(tiempos[k])))
        print(f"{k:13s} media={v.mean():8.2f} desv={v.std():6.2f} mejor={v.min():8.2f} peor={v.max():8.2f} "
              f"éxito(<=1% ref)={np.mean(v <= ref_f * 1.01):.0%} tiempo={np.mean(tiempos[k]) * 1000:6.1f} ms")
    Hm = sensibilidad(); etas, out = sensibilidad_eta()
    graficas(res, curvas, ref_x, ref_f, sol, Hm); graf_eta(etas, out)
    print("Sensibilidad T0 x alpha (costo medio):\n", np.round(Hm[2], 1))
    print("Sensibilidad eta:", {k: [round(v, 1) for v in vs] for k, vs in out.items()})
    json.dump(dict(ref=ref_f, ref_x=ref_x.tolist(), resumen=resumen, Ts=list(Hm[0]), alphas=list(Hm[1]),
M=Hm[2].tolist(),
                   etas=list(etas), eta_out=out, nombres=NOMBRES, pesos=W.tolist(), n_semillas=30, n_iter=3000),
              open("resultados_s3.json", "w"), indent=1)
```

Salida real de la ejecución:

```
Referencia global (evolución diferencial): f = 1318.72
HC            media= 1377.66 desv=116.83 mejor= 1322.26 peor= 1642.03 éxito(<=1% ref)=83% tiempo=  43.4 ms
HC reinicios  media= 1326.21 desv=  1.64 mejor= 1322.83 peor= 1330.72 éxito(<=1% ref)=100% tiempo=  43.4 ms
SA            media= 1325.66 desv=  1.95 mejor= 1321.89 peor= 1328.95 éxito(<=1% ref)=100% tiempo=  44.4 ms
Sensibilidad T0 x alpha (costo medio):
 [[1404.2 1377.1 1377.1 1378.1]
 [1377.7 1325.9 1325.  1326.7]
 [1325.4 1325.4 1326.1 1326.3]
 [1325.5 1324.8 1326.9 1329.8]
 [1325.8 1325.5 1325.8 1336. ]]
Sensibilidad eta: {'HC': [1460.1, 1461.3, 1427.8, 1430.0, 1332.0, 1348.8], 'SA': [1383.2, 1342.5, 1323.4, 1325.4,
1333.3, 1350.6]}
```

Capítulo 8. Análisis y discusión

8.1 Hill Climbing frente a Simulated Annealing

Hill Climbing alcanzó un costo medio de 1377.7 ton·km con desviación de 116.8; en 5 de 30 ejecuciones quedó atrapado en un óptimo local (peor costo 1642, 25 % sobre la referencia). Simulated Annealing logró 1325.7 ± 2.0 y 100% de éxito, es decir, resultados más estables a igual costo computacional (44 ms por corrida). En el paisaje de costo (sección 10) se observa un mínimo local aislado en la zona de Cuautla, que explica los fallos: cuando un centro cae en esa cuenca, ningún vecino cercano mejora y Hill Climbing se detiene.

8.2 Hill Climbing con reinicios

Repartir el mismo presupuesto en 10 reinicios dio 1326.2 ± 1.6, prácticamente igual a Simulated Annealing (diferencia de 0.6 ton·km, menor que una desviación estándar). Con solo 4 variables y un paisaje con pocos mínimos, los reinicios bastan; la ventaja de Simulated Annealing debería notarse más en problemas con muchos más óptimos locales.

8.3 Sensibilidad a los parámetros

Con T_0 bajo (5) el recocido casi no acepta empeoramientos y se parece a Hill Climbing; con $T_0 \geq 20$ y $\alpha \geq 0.995$ el costo medio se estabiliza cerca de 1,325. Un enfriamiento demasiado rápido ($\alpha = 0.990$ con $T_0 = 20$) pierde exploración, y uno demasiado lento ($T_0 = 400$, $\alpha = 0.9995$) termina todavía caliente tras 3,000 iteraciones y

empeora a 1,336. El paso η también importa: Simulated Annealing fue mejor con η entre 2 y 3 km; con pasos pequeños Hill Climbing casi no sale de su cuenca.

8.4 Limitaciones

- Las demandas y la red de distancias son ilustrativas (distancia euclidiana, sin carreteras ni capacidades).
- La referencia global proviene de otro algoritmo estocástico; es muy probable que sea el óptimo, pero no está demostrado.
- Se usó un presupuesto fijo de 3,000 iteraciones; con más iteraciones las diferencias podrían reducirse.

Capítulo 9. Conclusiones

- Hill Climbing es el más simple y rápido, pero en este problema falló en 5 de 30 ejecuciones por mínimos locales; Simulated Annealing y Hill Climbing con reinicios lograron 100% de éxito.
- Simulated Annealing es robusto si sus parámetros son razonables: la temperatura inicial y el enfriamiento deben ajustarse al presupuesto de iteraciones.
- Comparar con una referencia global y con varias semillas evita conclusiones basadas en una sola corrida afortunada.
- En problemas pequeños, estrategias simples (reinicios) compiten con métodos más sofisticados; elegir el algoritmo depende del tamaño y la forma del paisaje.

Capítulo 10. Plus: laboratorio de sensibilidad y paisaje de costo

Para entender *por qué* funcionan los algoritmos, se añadieron tres análisis: el paisaje de costo (muestra los mínimos locales), el mapa de calor de $T_0 \times \alpha$ y el efecto del paso η.

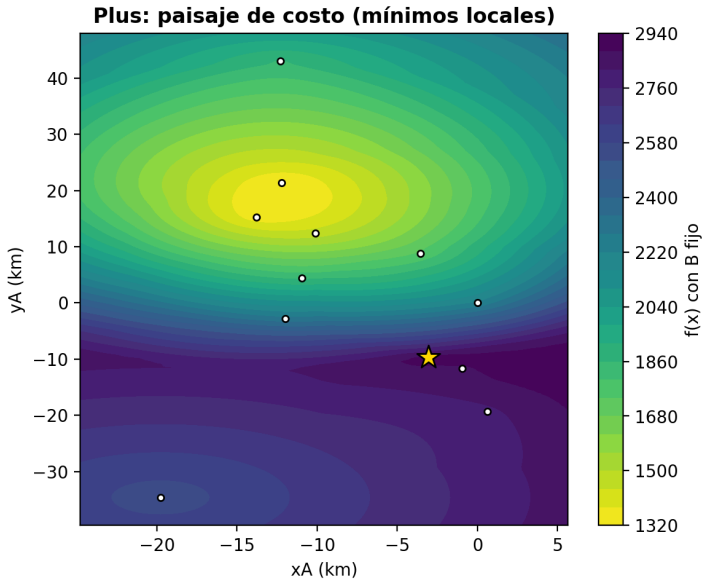


Figura 4. Costo al mover el centro A con el centro B fijo en la referencia; se aprecia una cuenca secundaria en el extremo suroeste (Cuautla).

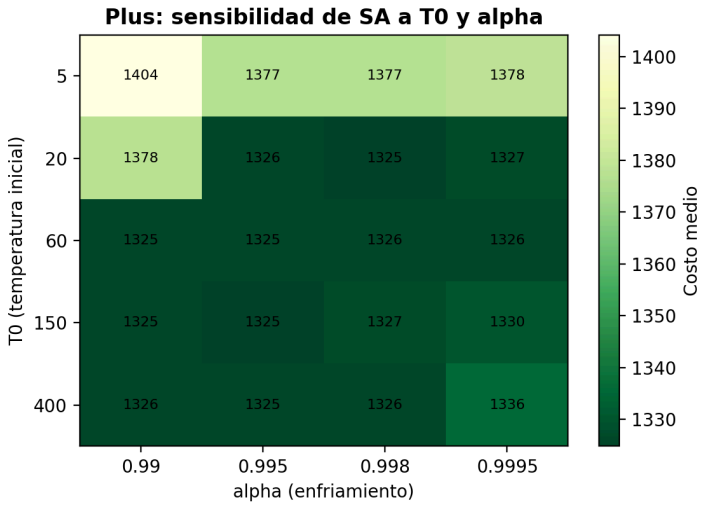


Figura 5. Costo medio de Simulated Annealing (12 semillas) según T_0 y α.

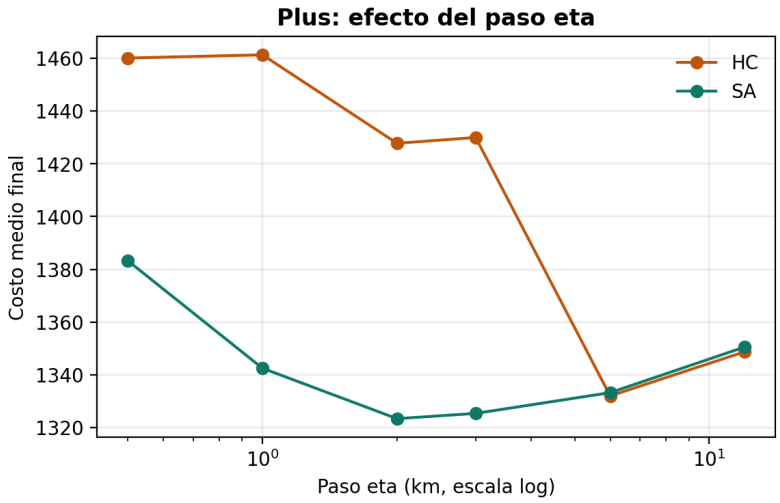


Figura 6. Costo medio final según el paso η (15 semillas).

Capítulo 11. Referencias (APA 7.ª ed.)

Kirkpatrick, S., Gelatt, C. D., Jr., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., & Teller, E. (1953). Equation of state calculations by fast computing machines. *The Journal of Chemical Physics, 21*(6), 1087–1092. https://doi.org/10.1063/1.1699114

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Russell, S., & Norvig, P. (2020). *Inteligencia artificial: Un enfoque moderno* (4.ª ed.). Pearson.

Storn, R., & Price, K. (1997). Differential evolution: A simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization, 11*(4), 341–359. https://doi.org/10.1023/A:1008202821328

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods, 17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wirsansky, E. (2020). *Hands-on genetic algorithms with Python*. Packt.

Anexo A. Lista de cotejo (S3)

#	Criterio de evaluación	S3
1	Portada, tema, sección y datos del problema completos en el PDF	✓
2	Planteamiento, formulación y procedimiento claramente explicados	✓
3	Código Python funcional y comentado, probado con datos de entrada	✓
4	Solución enmarcada, con gráficos, tablas o diagramas de resultados	✓
5	Repositorio GitHub con carpeta del alumno, README y PDFs	Pendiente: subir a GitHub
6	Declaración de IA utilizada, referencias APA 7 y conclusiones reflexivas	✓

Declaración de uso de IA

Herramienta: Claude (Anthropic), asistente conversacional.
Prompt: Revisa la actividad y verifica que todo este bien, modifica donde sea necesario, dale formato al archivo y diseño.
Uso: apoyo para proponer el problema, escribir el código de Hill Climbing y Simulated Annealing, ejecutar los experimentos y redactar el borrador.
Verificación: el código se ejecutó y su salida se incluye en el capítulo 7; las cifras del texto se calcularon desde el archivo de resultados; las referencias se tomaron de su DOI. Revisión personal por el alumno: [fecha].